GATE

Be sure with Vidyalankar Since 1960

Pearl Centre, S.B. Marg, Dadar (W), Mumbai – 400 028. Tel. 4232 4232

# CS : COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

## Digital Logic and Computer Organization and Architecture

## INDEX


| Sr. No. | Contents | Topics | Pg. No. |
|---|---|---|---|
| **1.** | **Digital Circuits** | | |
| | Notes | Number System | 1 |
| | | Codes | 8 |
| | | Boolean Algebra | 11 |
| | | Logic Gates | 11 |
| | | Karnaugh Map (K–MAP) | 21 |
| | | List of Formulae | 27 |
| | | LMR (Last Minute Revision) | 29 |
| | Assignment–1 | Questions | 33 |
| | Test Paper–1 | Questions | 36 |
| **2.** | **Combinational Logic Circuits, MUX and Decoders** | | |
| | Notes | Implementing Combinational Logic | 39 |
| | | Functions of Combinational Logic | 41 |
| | | Comparators | 44 |
| | | Multiplexers | 48 |
| | | Demultiplexers | 53 |
| | | Decoders | 55 |
| | | Encoders | 57 |
| | | Code Converters | 59 |
| | | LMR (Last Minute Revision) | 61 |
| | Assignment–2 | Questions | 63 |
| | Test Paper–2 | Questions | 68 |
| **3.** | **Sequential Logic Circuits** | | |
| | Notes | Introduction | 72 |
| | | Classification of Sequential Circuits | 72 |
| | | Flip Flop | 73 |
| | | Clocked SR Flip Flop | 74 |

| Sr. No. | Contents | Topics | Pg. No. |
|---|---|---|---|
| | | Preset and Clear | 75 |
| | | J – K Flip Flop | 76 |
| | | Master Slave J – K Flip Flop | 78 |
| | | D – Flip Flop | 79 |
| | | T – Flip Flop | 80 |
| | | Excitation Table of Flip Flop | 80 |
| | | Conversion from One Type of Flip Flop to Another | 81 |
| | | Important Parameters of Flip Flop | 82 |
| | | Applications of Flip Flop | 83 |
| | | Clocked Sequential Circuit | 87 |
| | | LMR(Last Minute Revision) | 87 |
| | Assignment–3 | Questions | 88 |
| | Test Paper–3 | Questions | 92 |
| **4.** | **Overview of Computer System** | | |
| | Notes | Introduction | 95 |
| | | Numbers and Arithmetic Operations | 97 |
| | | Bit Slice Processor | 100 |
| | | Basic ALU Organization | 101 |
| | | Booth's Algorithm | 110 |
| | | IEEE Standard for Floating Point Numbers | 116 |
| | | IEEE Standard for Binary Floating Point Arithmetic | 120 |
| | | LMR (Last Minute Revision) | 122 |
| | Assignment–4 | Questions | 127 |
| | Test Paper–4 | Questions | 132 |
| **5.** | **Instructions** | | |
| | Notes | Instruction Cycle | 134 |
| | | Addressing Modes | 136 |
| | | Instruction Formats | 139 |
| | | Instruction Interpretation | 141 |
| | | Implementation Method | 141 |
| | | LMR (Last Minute Revision) | 147 |
| | Assignment–5 | Questions | 149 |
| | Test Paper–5 | Questions | 153 |

| Sr. No. | Contents | Topics | Pg. No. |
|---|---|---|---|
| **6.** | **Memory Organization, I/O, Serial Communication** | | |
| | Notes | Introduction | 156 |
| | | Memory Hierarchy | 156 |
| | | Memory Characteristics | 161 |
| | | Semiconductor RAM Memories | 162 |
| | | Static Memories | 163 |
| | | CMOS Cell | 164 |
| | | Static RAM | 165 |
| | | Designing Examples | 167 |
| | | Read Only Memories | 169 |
| | | Cache Memory Architecture and Working | 171 |
| | | Mapping Methods | 173 |
| | | Virtual Memory Technology | 181 |
| | | Secondary Memory Technology | 188 |
| | | Input and Output Unit | 192 |
| | | Direct Memory Access | 196 |
| | | Bus Arbitration | 211 |
| | | Multiple Processor Organisation | 218 |
| | | Parallel Processing Applications | 221 |
| | | Solved Examples | 226 |
| | | LMR (Last Minute Revision) | 228 |
| | Assignment –6 | Questions | 238 |
| | Test Paper – 6 | Questions | 241 |
| **ID Problems** | | Questions | 244 |
| **Practice Problems** | | Questions | 248 |
| **SOLUTIONS** | | | |
| **Assignment** | | Answer Key | 263 |
| | | Model Solutions | 265 |
| **Test Paper** | | Answer Key | 282 |
| | | Model Solutions | 284 |
| **ID Problems** | | Answer Key | 299 |
| | | Model Solutions | 300 |
| **Practice Problems** | | Answer Key | 303 |
| | | Model Solutions | 304 |

# Topic 1 : Digital Circuits

## NUMBER SYSTEM

### I. Binary Numbers

The binary number system with two digits is a base two system. The two binary digits are 0 and 1. The position of 1 or 0 in a binary number indicates its weight or value within the number. The weight of each successive higher position in the binary number is an increasing power of two.

Using n bits we can count upto a decimal number $(2^n - 1)$.

- **Binary Arithmetic**

(a) Additions — Rules of addition

$0 + 0 = 0$
$0 + 1 = 1$
$1 + 0 = 1$
$1 + 1 = 10$

Carry Sum

(b) Subtraction — Rules of Subtraction

$0 - 0 = 0$
$1 - 0 = 1$
$1 - 1 = 0$
$0 - 1 = 11$

Borrow Sub.

(c) Multiplication — Rules of Multiplication

$0 \times 0 = 0$
$0 \times 1 = 0$
$1 \times 1 = 1$

- **1's Complement of a Binary Number**

The 1's complement of a binary number is found by simply changing all 1's to 0's and all 0's to 1's.

| | Binary No. | | 1's Complement |
|---|---|---|---|
| | 10101 | | 01010 |
| e.g.: | (a) 1011 | → | 1's complement is 0100 |
| | (b) 0101 | → | 1's complement is 1010 |

**1's Complement Subtraction**

Using 1's complement we can subtract two binary number by means of addition. To subtract a smaller number from a larger number the 1's complement method is as follows:

(i) Determine 1's complement of the smaller number
(ii) Add 1's complement to the larger number

(iii) Remove the carry and add it to the result. This is called end around carry.
e.g. Subtract $(10011)_2$ from $(11001)_2$

(a) Direct Subtraction :
$$\begin{array}{r} 11001 \\ -\ 10011 \\ \hline \end{array}$$
Ans. : $(00110)_2$

(b) Using 1's Complement:
$$\begin{array}{r} 11001 \\ +\ 01100 \\ \hline 1\ 00101 \\ \text{Carry} \longrightarrow\ 1 \\ \hline \end{array}$$
$\rightarrow$ is complement of 10011

Ans. : $(00110)_2$

To subtract a larger number from smaller number 1's complement method is as follows:

(i) Determine the 1′s complement of the larger number.
(ii) Add the 1′s complement to the small number
(iii) The answer has an opposite sign and is the 1′s complement of the result. There is no carry.

e.g.: Subtract $(1101)_2$ from $(1001)_2$. Using 1's complement.

Direct Subtraction:
$$\begin{array}{r} 1001 \\ -\ 1101 \\ \hline \end{array}$$
Ans.: $-0100$

Using 1's complement:
$$\begin{array}{r} 1001 \\ +\ 0010 \\ \hline \end{array}$$
$1011 \rightarrow$ This is 1′s complement from and opposite in sign.

Ans.: $-0100$. Take the 1′s complement.

- **2′s Complement of a Binary Number**

The 2's complement can be obtained by adding 1 to the 1's complement of the binary number.
e.g.: $(1010)_2$ . Find 2's complement

| | | |
|---|---|---|
| 1's complement = | | 0101 |
| Add1 | | + 1 |
| Ans. | : | 0110 |

- **To find 2's complement of a number**

Start from the right and write the bits as they are upto and including the first 1. Then take the 1's complement of the remaining bits.

e.g.: $(1001101100)_2$ Find 2′s complement

$\underline{0110010}$ $\underline{100}\rightarrow$ same upto first 1, going right to left

↓ ↓

1's complement of remaining bits.

### 2's Complement Subtraction

To subtract a smaller number from a larger one, the 2's complement method is as follows:

(i) Determine the 2's complement of smaller number
(ii) Add the 2's complement to the larger number
(iii) Discard the carry. (Always there is a carry)

e.g.: Find 1100 – 1011

| | | | |
|---|---|---|---|
| 1100 | | 1100 | |
| – 1011 | | + 0101 | 2's complement |
| (0001) | | 1 0001 | |
| Direct method | | Discard the carry | |

Ans.: 0001.

To subtract a larger number from a smaller number we use the following 2's complement method:

(i) Determine 2's complement of larger number
(ii) Add the 2's complement to the smaller number
(iii) To get the answer take the 2's complement and change the sign of the above result.

e.g.: Find 10011 – 11100

Direct Method :

10011
– 11100
– 01001

Using 2's complement :

10011
+ 00100
10111 No carry, hence 2's complement of answer.

∴ Answer = (– 01001).

**Note**:
Both the 1's and 2's complement are complex compared to direct subtraction. But they have distinct advantage when implemented using logic circuits because they allow subtraction to be done by using only addition. Both 1's and 2's complement can be realized using logic circuits and 2's complement has an advantage over the 1's complement in that an end around carry operation does not have to be performed.

### Binary to Decimal Conversion

A binary number is a weighted number. The value of a given binary number in terms of its decimal equivalent can be determined by adding the products of each bit and its weight. The right most bit is the Least Significant Bit (LSB) in the binary number and has a weight of $2^0 = 1$. The weight increases by a power of 2 for each bit from right to left

| | | | | | |
|---|---|---|---|---|---|
| Binary weight | $2^4$ | $2^3$ | $2^2$ | $2^1$ | $2^0$ |
| Weighted value | 16 | 8 | 4 | 2 | 1 |
| Binary No. | (1 | 0 | 1 | 1 | $1)_2$ |

$1 \times 16 + 0 \times 8 + 1 \times 4 + 1 \times 2 + 1 \times 1 = (23)_{10}$

Fractional number can also be represented in binary by placing bits to the right of the binary point.

The column weights of a binary number are:

$2^n \ldots 2^3 2^2 2^1 2^0 . 2^{-1} 2^{-2} 2^{-3} \ldots 2^{-n}$

↓

binary point

e.g.: (a) $(1011)_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 8 + 0 + 2 + 1 = (11)_{10}$

(b) $11.01 = 1 \times 2^1 + 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-1} + 1 \times 2^{-2} = 2 + 1 + \frac{1}{4} = (3.25)_{10}$

**Binary to Decimal Conversion :** (Dibble Dabble Method)

(i) Start with the MSB and multiply by 2.
(ii) Add the next bit to the product.
(iii) Multiply the sum by 2.
(iv) Add the next bit to the sum.
(v) Multiply by 2 and repeat the steps until all the bits are exhausted.

e.g.: (a) $(1011)_2$

Answer: $1 \times 2 = 2$

$2 + 0 = 2$

$2 \times 2 = 4$

$4 + 1 = 5$

$5 \times 2 = 10$

$10 + 1 = 11$

Answer: $(11)_{10}$

(b) $(1111)_2$

Answer: $1 \times 2 = 2$

$2 + 1 = 3$

$3 \times 2 = 6$

$6 + 1 = 7$

$7 \times 2 = 14$

$14 + 1 = 15$

Answer: $(15)_{10}$

**Decimal to Binary Conversion :**

(i) Sum of weight method – by placing 1 in the appropriate weight position and 0 in other position.

| Remember: … | $2^5$, | $2^4$, | $2^3$, | $2^2$, | $2^1$, | $2^0$ | |
|---|---|---|---|---|---|---|---|
| … | 32 | 16 | 8 | 4 | 2 | 1 | Binary weights. |

(ii) By repeated division method (Double Dabble Method)
We begin by dividing the given decimal number by 2 and then dividing each resulting quotient by 2 until there is 0 quotient. The remainder generated by each division form the binary number. The 1st remainder is the LSB of the binary number.

e.g. $(28)_{10} = (11100)_2$

| | | |
|---|---|---|
| 2 | 28 | |
| 2 | 14 | 0 |
| 2 | 7 | 0 |
| 2 | 3 | 1 |
| 2 | 1 | 1 |
| | 0 | 1 |

To convert the decimal fractions to binary we use repeated multiplications by 2. We begin multiplying the given fractional number by 2 and then multiplying each resulting fractional part by 2 until the fractional part is zero. The carry generated by each multiplication form the binary number. The 1st carry produced is the MSB.

e.g.: $(0.3125)_{10} \rightarrow (0.0101)_2$

**II. Octal Number System** (Base of 8)

Older computer systems use octal numbers to represent binary information. There are eight combinations of 3 bit binary number. Therefore, sets of 3 bit binary numbers can be represented by octal numbers and this can be conveniently be used for entering data in the computer. Octal number system uses eight symbols 0 to 7. Octal numbers are also referred to as base 8 numbers. The advantage of the octal system is its usefulness in converting directly from a 3 bit binary number.

**Octal to Decimal Conversion**

| Weight | $8^3$ | $8^2$ | $8^1$ | $8^0$ |
|---|---|---|---|---|
| Decimal value | 512 | 64 | 8 | 1 |
| Octal No. | 2 | 3 | 7 | 4 |

$(2374)_8 = (1276)_{10}$

$2 \times 512 + 3 \times 64 + 7 \times 8 + 4 \times 1 = 1276$

Since the octal number system has base of 8 each successive digit position is an increasing power of 8, beginning with the eight most column with 8°. The decimal number can be obtained by multiplying each digit by its weight and summing the products.

**Decimal to Octal Conversion**

To get octal equivalent of a decimal number we use repeated division by 8. The decimal number is divided by right, the quotient is divided by 8 and the remainders obtained will give the octal number. The first remainder is the LSB.

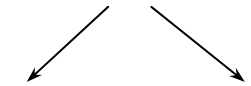
**Octal to Binary Conversion**

The primary application of octal number is in the representation of binary number since it takes only one octal digit to represent three bits octal number are much easier to read, than binary number.

| **Octal digit** | **Binary number** |
|---|---|
| 0 | 000 |
| 1 | 001 |
| 2 | 010 |
| 3 | 011 |
| 4 | 100 |
| 5 | 101 |
| 6 | 110 |
| 7 | 111 |

To convert octal to binary, simply replace the octal digit by its binary number using 3 bits.

e.g.: $(47)_8$ → Octal No.



$(100)_2$ $(111)_2$

$\therefore$ $(47)_8 = (100111)_2$

**Binary to Octal Conversion**

Break the binary number into groups of 3 bits and convert each group into an appropriate octal digit.

e.g.: $(100\ 111)_2$ → Binary

4 7

$\therefore$ $(100111)_2 = (47)_8$.

**III. Hexadecimal No. System** (Base of 16)

Computers require binary data, but people working with computers have trouble remembering long binary words. One solution to the problem is to use hexadecimal or base – 16 number system. Hex is more compact than decimal, two hexadecimal digits can represent a decimal number upto 255 Each hex digit is equal to 4 binary digits.

The hexadecimal system has a base of 16 i.e. it is composed of 16 digits and characters. Ten digits and six alphabetic characters make up this system 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F.

| Decimal | Binary | Hexadecimal |
|---|---|---|
| 0 | 0000 | 0 |
| 1 | 0001 | 1 |
| 2 | 0010 | 2 |
| 3 | 0011 | 3 |
| 4 | 0100 | 4 |
| 5 | 0101 | 5 |
| 6 | 0110 | 6 |
| 7 | 0111 | 7 |
| 8 | 1000 | 8 |
| 9 | 1001 | 9 |
| 10 | 1010 | A |
| 11 | 1011 | B |
| 12 | 1100 | C |
| 13 | 1101 | D |
| 14 | 1110 | E |
| 15 | 1111 | F |

**Binary to Hexadecimal Conversion**

Break the binary number in four bit groups starting at the binary point and replace each group with the equivalent hexadecimal symbol.

e.g. $(11011001)_2$
D 9
$\therefore$ $(1101\ 1001)_2 = (D9)_{16}$.

**Hexadecimal to Binary Conversion**

Replace each hexadecimal digit by the four bit binary number.

e.g. $(4CA)_{16} = (0100\ 1100\ 1010)$
4 C A

With 2 hexadecimal digits, we can count upto $(FF)_{16} = (256)_2$, with four hexadecimal digits, we get $(FFFF)_{16} = (65536)_{10}$.

**Hexadecimal to Decimal Conversion**

To get the decimal equivalent, multiply each hexadecimal digit by its weight and sum all the products.

| Hexadecimal weight | $16^3$ | $16^2$ | $16^1$ | $16^0$ |
|---|---|---|---|---|
| Weight value | 4096 | 256 | 16 | 1 |
| Hexadecimal No. | B | 2 | F | 8 |

$(B2F8)_{16} = B \times 4096 + 2 \times 256 + F \times 16 + 8 + 1 = (45816)_{10}$

OR

We can convert the hexadecimal number to the binary number and then convert to decimal number.

**Decimal to Hexadecimal Conversion**

Repeated division of a decimal number by 16 gives the hexadecimal number, formed by the remainders of each division. The 1st remainder is the LSB of the hexadecimal number.

**Hexadecimal Addition**

The following rules are applied :

1. In any given column of an addition problem, think of the two hexadecimal digits in terms of their decimal value.
   e.g. $(5)_{16} = (5)_{10}$
   $(C)_{16} = (12)_{10}$
2. If the sum of these two digits is $(15)_{10}$ or less, bring down the corresponding hexadecimal digit.
3. If the sum of these two digits is greater than $(15)_{10}$, bring down the amount of the sum that exceeds $(16)_{10}$ and carry a 1 to the next column.

   e.g. $(DF)_{16} + (AC)_{16}$

```
   D   F
+  A   C
  18   B
```

Procedure :

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| | | F | + | C | | = | 15 + 12 = 27 |
| | ∴ | 27 | – | 16 | | = | B with carry 1 |
| | | D | + | A | + 1 | = | 13 + 10 + 1 = 24 |
| | | 24 | – | 16 | | = | 8 with a carry 1. |

## CODES

Almost all digital circuits (Computers, calculators) understand only binary numbers. But most people understand only decimal numbers. Thus, we must have electronic devices that can translate from decimal to binary and from binary to decimal numbers.

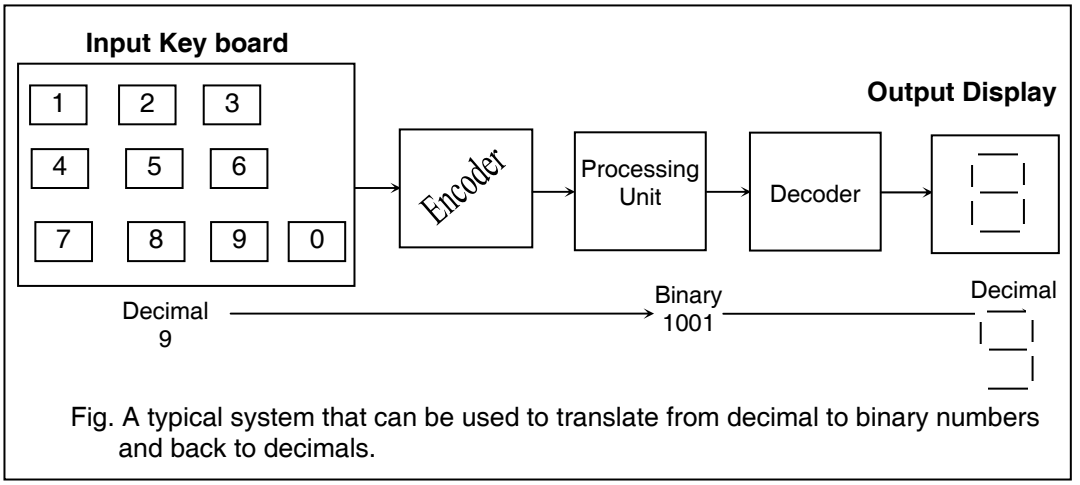



Fig. A typical system that can be used to translate from decimal to binary numbers and back to decimals.

The device that translates from keyboard decimal numbers to binary is called an *encoder*. The device converting from binary numbers to decimal numbers is called a *decoder.*

**Binary Codes**

In this, the decimal numbers are converted, to their binary equivalent.
e.g. 13 is represented as 1101.

**8–4–2–1 (BCD) Code**

Decimal digits 0 through 9 are represented by their binary equivalents using four bits. Remaining numbers are considered forbidden numbers.

In applications such as frequency counters, digital voltmeters or calculators where the output is decimal display, BCD code is usually used.

| e.g. | 5 | 2 | 9 |
|---|---|---|---|
| | 0101 | 0010 | 1001 |

**Excess –3**

The term BCD is a general term usually referring to an 8421 code. Another code that is really a BCD code, is the excess 3 code. The code can be derived from the BCD by adding 3 to each coded number.

| e.g., | 3 | 8 |
|---|---|---|
| | + 3 | + 3 |
| | 6 | 11 |
| | 0110 | 1011 |

The excess – 3 code is used in many arithmetic circuits because it is self complementing. It is useful when it is desired to obtain nine's complement of a decimal digit represented by this code. The nine's complement is obtained by simply complementing each bit.

e.g. 9's complement of 4 (0111 is X–3 code) is 1000 in X–3. This helps considerably in performing subtraction operation in digital computers.

**Alphanumeric Codes**

When communicating with or between computers, data may consist of numerals, letters and special symbols. We require a binary–base code which can represent letters of the alphabet as well as numbers. If we use n bit binary code, we can represent $2^n$ elements using this code. Therefore to represent 10 digits 0 through 9 and 26 alphabets A to Z, we need minimum 6 bit alphanumeric code. ($2^6 = 64$). 6 bit alphanumeric code is used in many computers to represent alphanumeric characters and symbols internally and therefore can be called internal code. Frequently there is a need to represent more than 64 characters including the lower case letters and special control characters for the transmission of digital information. For this reason the following two codes are normally used.

1. Extended BCD Interchange Code (EBCDIC)
2. ASCII

**EBCDIC**

This is an 8 bit code without parity. A ninth bit can be added for parity. It is used in IBM equipments.

**ASCII**

This code is widely used to send information to and from microcomputers. It is a 7 bit code used in transferring coded information from keyboards and to computer displays and printers. ASCII stands for the American Standard Code for Information Interchange. The ASCII Code is used to represent numbers, letters, punctuation marks as well as control characters. e.g. The 7 bit ASCII Code 111 1111 stands for DEL, i.e., Delete. With 7 bits we can code upto 128 characters which is enough for the full upper–and lower case alphabet, numbers, punctuation marks, and control characters. The code is arranged so that if only uppercase letters, numbers, and a few control characters are needed, only the lower 6 bits are all that are required. If a parity check is wanted, a parity bit is added to the basic 7 bit code in the MSB position. The binary word 1100 0100 is the ASCII Code for uppercase D with odd parity.

**Hollerith Code**

Many large computers use punched cards and card readers, which read data from the cards. When a hole is punched into a card, a beam of light can pass through the hole and is read as 1. A card consists of 80 columns and 12 rows. Each column represents an alphanumeric character with holes in the appropriate rows. A hole is sensed as 1 and absence of a hole is sensed as O by the circuits in the Card Reader.

The Rows are marked starting from the top as 12, 11, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9. The first three are called zone punch and the last 9 are called the numeric punch rows. The code used in this is known as Hollerith Code.

## Error Detecting and Error Correcting Codes

When the digital information in the binary form is transmitted from one circuit or system to another circuit or system an error may occur. This means a signal corresponding to O may change to 1 and vice–versa.

### Parity

Parity is error detecting technique. To detect errors a constant check of transmitted data is done. To check accuracy an extra parity bit is generated and transmitted. By parity we mean number of 1's in a digital data which may be even (even parity) or odd parity. The data along with parity bit is transmitted. The parity of the received data is checked. If the odd number of data bit changes, the parity check gives an error. The even numbers of change in data bit won't affect the parity, hence error cannot be detected.

### Hamming Code

The hamming code is an error detecting and correcting code. Along with data bits a number of parity bits are sent. The bit positions are reserved for the parity bits and data bits.

e.g., consider a 4 bit data transmitted with 3 parity bits, then entire transmitted word appears as,

| Position value | $2^3$ | $2^2$ | $2^1$ | | $2^0$ | | |
|---|---|---|---|---|---|---|---|
| Bit position | 7 | 6 | 5 | 4 | 3 | 2 | 1 |
| Bit value | $D_7$ | $D_6$ | $D_5$ | $P_4$ | $D_3$ | $P_2$ | $P_1$ |

$P_1$ Gives even parity over bits 1, 3, 5 and 7
$P_2 \rightarrow$ 2, 3, 6 and 7
$P_4 \rightarrow$ 4, 5, 6 and 7

suppose that a data word 1010 is transmitted. The Hamming code would be

| 7 | 6 | 5 | 4 | 3 | 2 | 1 |
|---|---|---|---|---|---|---|
| 1 | 0 | 1 | $P_4$ | 0 | $P_2$ | $P_1$ |

even parity for $P_1$ gives (1, 3, 5, 7) = $P_1 + 0 + 1 + 1 = P_1 + 2$

$\therefore \quad P_1 = 0$

$P_2$ gives (2, 3, 6, 7) = $P_2 + 0 + 0 + 1 = P_2 + 1$

$\therefore \quad P_2 = 1$

$P_3$ gives (4, 5, 6, 7) = $P_4 + 1 + 0 + 1 = P_4 + 2$

$\therefore \quad P_4 = 0$

Hence transmitted word could be

| $D_7$ | $D_6$ | $D_5$ | $P_4$ | $D_3$ | $P_2$ | $P_1$ |
|---|---|---|---|---|---|---|
| 1 | 0 | 1 | 0 | 0 | 1 | 0 |

1101 Ans : $\rightarrow$1 1 0 0 1 1 0

### Error Correction

Parity bits are checked for even parity suppose $D_6$ is lost i.e., $D_6$ changes to 0 instead of 1.

$\therefore$ even parity for $P_4$, $P_4 + D_5 + D_6 + D_7 = 0 + 0 + 0 + 1$

$\therefore \quad C_1 = 1$ (error present) $\qquad C_2 = 1$

for $P_2$, $\quad P_2 + D_3 + D_6 + D_7 = 1 + 1 + 0 + 1 = 1$ error present $\quad C_3 = 0$

for $P_1$, $\quad P_1 + D_3 + D_5 + D_7 = 0 + 1 + 0 + 1 = 0$ No error

$\therefore \quad C_1, C_2, C_3 = 110 = D_6$ has error.

## BOOLEAN ALGEBRA

Boolean Algebra is the mathematics of digital system. The objective of the use of the Boolean algebra are :

i. To simplify the procedure necessary to solve logical problems.
ii. To simplify any circuit, by using fewer components to perform a function.

Boolean algebra is a system of mathematical logical which differs from both ordinary algebra and the binary number system.

There are two constants within the Boolean system : 0 and 1. There are no fractional or negative numbers in boolean algebra. Boolean algebra uses two binary operations '+' (plus) and '.' (dot) and one unary operation '/' (complement). Variable, Complement and Literal are the terms used in boolean algebra.

*Variable* : A variable is a symbol used to represent a logical quantity that can have a value 1 or 0.

*Complement* : It is the inverse of a variable.

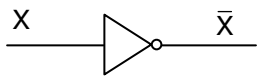
*Literal* : It is a variable or the complement of a variable

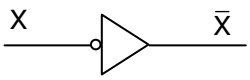
## LOGIC GATES

There are three types of logic functions, AND, OR, NOT.

**The NOT Gate**

The NOT gate (inverter) performs the operation called *inversion* or *complementation*. The inverter changes one logic level into other logic level i.e. it changes 0 to 1 and 1 to 0.




Standard Logic symbols of Inverter



For Inverter, when the input is LOW, the output is HIGH; when the input is HIGH, the output is LOW. The bubble always indicates inversion in digital circuit.

Truth Table

| I/P | O/P |
|---|---|
| 0 | 1 |
| 1 | 0 |

*Complementation laws*

$\overline{0} = 1$

$\overline{1} = 0$

$\overline{\overline{A}} = A$

**The AND Gate**

The AND function is mathematically represented by placing a dot between the two variables as $A \cdot B$




Standard Logic symbols for 2–input and 3–input AND gate

AND function is same as Boolean multiplication.

For a 2–input AND gate, the output X is HIGH only if both the inputs are HIGH.
The AND gate can have two or more inputs and performs logical multiplication.

Truth Table

| INPUTS | | OUTPUT |
|---|---|---|
| A | B | X |
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 0 | 0 | 1 |

The AND gate as an Enable/Inhibit Device : The AND gate is used to enable the passage of signal from one point to another at certain times and to inhibit the passage at other times.

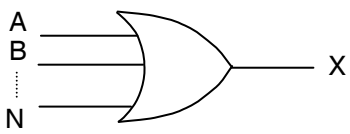
*Laws of AND function*

| | |
|---|---|
| $A \cdot 0 = 0$ | Null |
| $A \cdot 1 = A$ | Identity |
| $A \cdot A = A$ | Idempotent |
| $A \cdot \overline{A} = 0$ | Complement |
| $A \cdot B = B \cdot A$ | Commutative |
| $ABC = A(BC) = (AB)C$ | Associative |
| $A \cdot (B + C) = A \cdot B + A \cdot C$ | Distributive |

**The OR Gate**

The OR function is mathematically represented as

Y = A OR B OR C ……OR N
= A + B + C + ……..+ N




Standard Logic symbol for N input OR Gate

$X = A + B$ is read as "Y equals A OR B"



An OR gate produces a HIGH on the output when any of the inputs is HIGH. Boolean addition is the same as the OR function.

Truth table

| INPUTS | | OUTPUT |
|---|---|---|
| A | B | X |
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

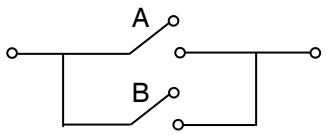
Boolean addition differs from binary addition in the case where two 1's are added. There is no carry in Boolean addition.

*Laws of OR function*

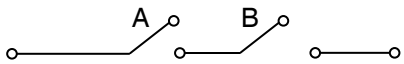
| | |
|---|---|
| $A + 0 = 0$ | Null |
| $A + 1 = A$ | Identity |
| $A + A = A$ | Idempotent |
| $A + \overline{A} = 1$ | Complement |
| $A + B = B + A$ | Commutative law |
| $A + (B + C) = (A + B) + C$ | Associative law |
| $(A + B)(A + C) = A + B \cdot C$ | Distributive law |

The OR gate is also called '*any or all*' gate.
The representation of OR, AND gate using switches

OR
Y = A + B




AND
Y = $A \cdot B$




*Operator Precedence*

When solving Boolean expressions, the precedence in descending order is

1. Parenthesis
2. NOT
3. AND
4. OR

*Absorption Laws*

$A(A + B) = A$

Proof :

$$A(A + B) = AA + AB$$
$$= A + AB$$
$$= A(1 + B)$$
$$= A$$

Similarly, $A + AB = A$

*Some rules of Boolean Algebra*

1. $\overline{\overline{A}} = A$
   The double complement of a variable is always equal to the variable.

2. $A + \overline{A}B = A + B$
   Proof :

$$\begin{aligned} A + \overline{A}B &= A + AB + \overline{A}B \\ &= AA + AB + \overline{A}B \\ &= AA + AB + A\overline{A} + \overline{A}B \\ &= (A + \overline{A})(A + B) \\ &= 1 \cdot (A + B) \\ &= A + B \end{aligned}$$

3. $\overline{A} + AB = \overline{A} + B$

4. $(A + B)(A + C) = A + BC$
   Proof :

$$\begin{aligned} (A + B)(A + C) &= AA + AC + AB + BC \\ &= A(1 + C) + AB + BC \\ &= A + AB + BC \\ &= A(1 + B) + BC \\ &= A + BC \end{aligned}$$

**De Morgan's Theorem**

It states that the complement of a function is obtained by interchanging AND and OR operators and complementing each literal.

*De–Morgan's Laws*

1. $\overline{A + B} = \overline{A} \cdot \overline{B}$
   NOR = bubbled AND

2. $\overline{AB} = \overline{A} + \overline{B}$
   NAND = bubbled OR

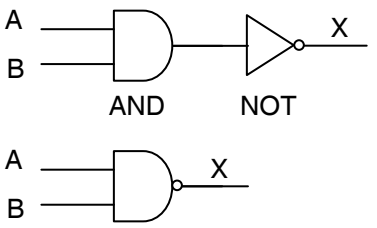


The NAND and NOR gates are called universal gates as any basic gate (AND, OR and NOT) can be implemented using these gates.

**The NAND Gate**

NAND operation is mathematically represented as

$$X = \overline{AB}$$

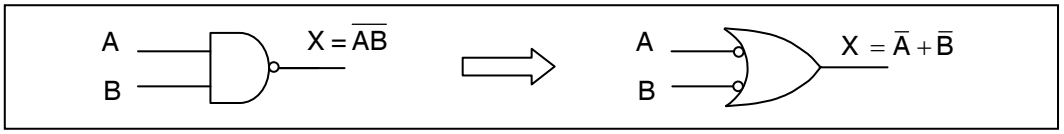



Standard Logic symbol for 2 input NAND Gate

By De–Morgan's law

$$\overline{AB} = \overline{A} + \overline{B}$$

i.e. NAND gate is equivalent to bubbled OR gate.

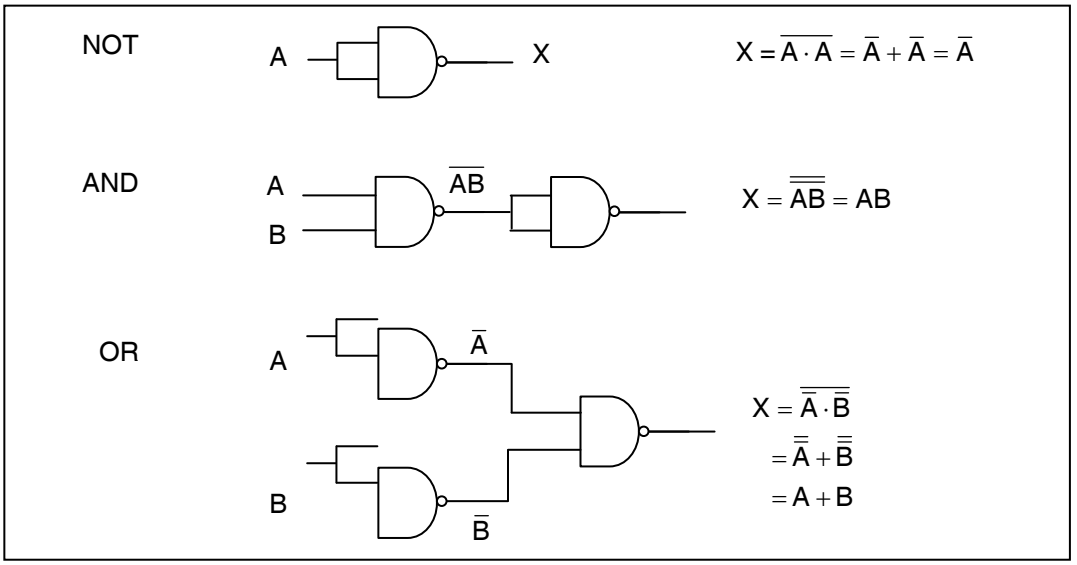



Truth table

| INPUTS | | OUTPUT |
|---|---|---|
| A | B | X |
| 0 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

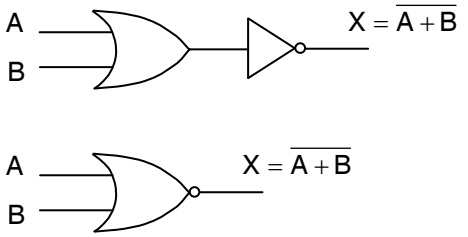
The output of NAND gate is HIGH, when at least one input to the NAND gate is LOW. All the basic gates can be represented using only NAND gate.

**Basic Gates using NAND Gates only**



### The NOR Gate

A NOT and OR gate combinely makes NOR gate.




Standard Logical symbol of 2–inputs NOR gate

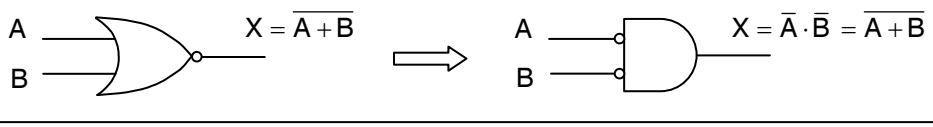
NOR gate is mathematically represented by

$$X = \overline{A + B}$$

By De Morgan's law

$$\overline{A + B} = \overline{A} \cdot \overline{B}$$

i.e. NOR gate is equivalent to bubbled AND gate.

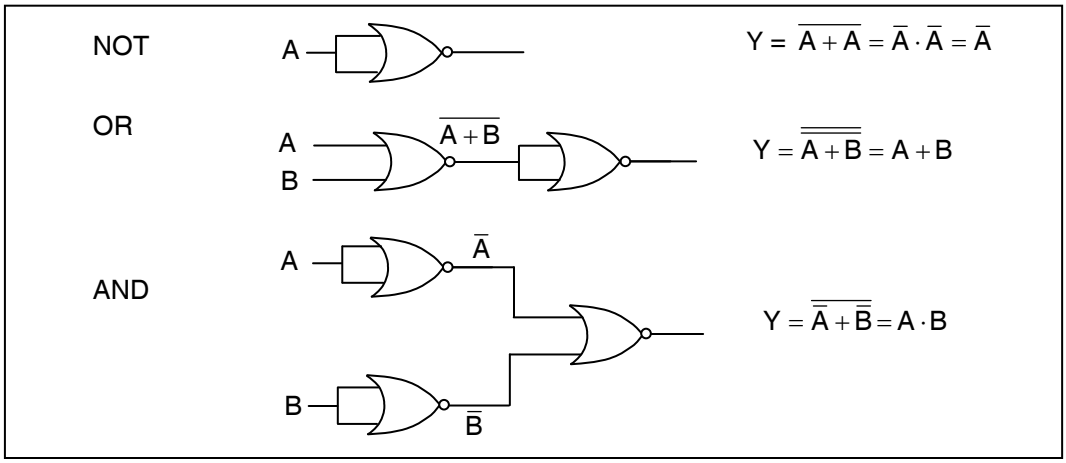



Truth Table

| INPUTS | | OUTPUT |
|---|---|---|
| A | B | X |
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 0 |

The output of NOR gate is HIGH only when all the inputs are LOW.

**Basic Gates using NOR Gates only**




The NAND and NOR functions are commutative but not associative.

i.e. $\overline{\overline{X + Y} + Z} \neq \overline{\overline{X + Z} + Y}$

Non associativity of NOR gate.

**Demorganization**

*Procedure*

Step 1 : Complement the entire function

Step 2 : Change all the ANDs to ORs and all the ORs to ANDs.

Step 3 : Complement each of the individual variables

Demorganizations can be stated in one line as
"*just break the line, change the sign*".

For example, demorganization of the function $y = \overline{\overline{A}B + C}$ is

1. complement function $\overline{y} = \overline{A}B + C$
2. change operators $\overline{y} = (\overline{A} + B)C$
3. complement variables $y = (A + \overline{B})\overline{C}$

**Converting Circuits to Universal Logic**

*Procedure*

Step 1: Draw the circuit in AND/OR/invert logic

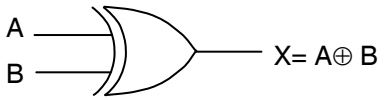
Step 2 : If NAND realization is chosen, add a circle to the outputs of each AND gate on the logic diagram, and add circles to the inputs of all OR gates.

Step 3 : In NOR realization, add circle to the output of gates and add circle to the inputs of AND gates.

Step 4 : Add or subtract an inverter on each line that received a circle in step 2 or 3.

**The Exclusive–OR Gate (EX–OR)**

The EX–OR is a modified form of OR gate. It is also called '*any but not all*' gate.




Standard Logical Symbol for 2–input EX–OR Gate

EX–OR function is represented by the symbol $\oplus$.

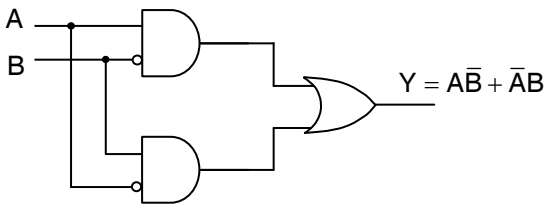
EX–OR is mathematically represented as

$$X = \overline{A}B + A\overline{B}$$

Truth Table

| INPUTS | | OUTPUT |
|---|---|---|
| A | B | X |
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

For even inputs output is LOW and for odd inputs output is HIGH.




It is also noted that EX–OR is inequality comparator.

The EX–OR operation is mathematically represented by following ways :

1. $X = A \oplus B$
2. $X = \overline{A}B + A\overline{B}$ ….SOP form
3. $X = (\overline{A} + B)(A + \overline{B})$ ….POS form
4. $X = (A + B)(\overline{A} + \overline{B})$ ….POS form
5. $X = \overline{AB + \overline{A}\overline{B}}$ ….SOP form
6. $X = (A + B)(\overline{A \cdot B})$ ….POS form

Let more than 2 inputs be EX–ORed,

$$A \oplus B \oplus C = (\overline{A}B + A\overline{B})\overline{C} + \overline{(\overline{A}B + A\overline{B})}C$$

$$= \overline{A}B\overline{C} + A\overline{B}\overline{C} + (\overline{\overline{A}B} \cdot \overline{A\overline{B}})C$$

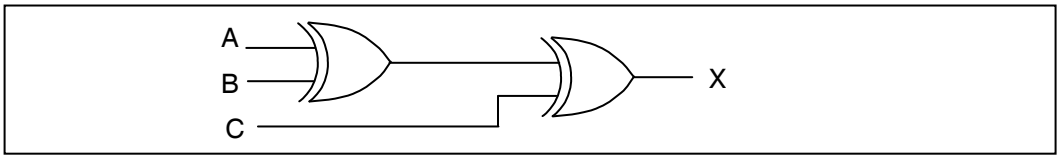
$$= \overline{A}B\overline{C} + A\overline{B}\overline{C} + ((A + \overline{B})(\overline{A} + B))C$$

$$= \bar{A}B\bar{C} + A\bar{B}\bar{C} + (A\bar{A} + AB + \bar{A}\bar{B} + B\bar{B})C$$
$$= \bar{A}B\bar{C} + A\bar{B}\bar{C} + ABC + \bar{A}\bar{B}C$$
$$= B(\bar{A}\bar{C} + AC) + \bar{B}(A\bar{C} + \bar{A}C)$$
$$= BC(A \odot C) + \bar{B}(A \oplus C)$$

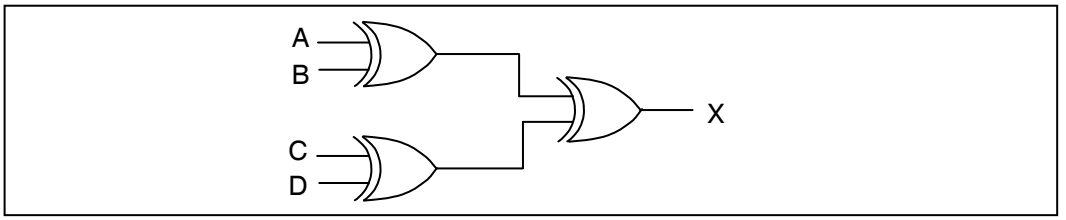
*Note: There are only 2–input EX–OR gates available in the market.*

To implement EX–OR function of more than 2–inputs following methods are used.

1. $X = A \oplus B \oplus C$




2. $X = A \oplus B \oplus C \oplus D$



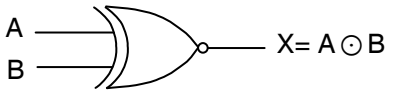

*Useful formulae for EX–OR function*

- $A \oplus A = A\bar{A} + \bar{A}A = 0$
- $A \oplus 0 = A\bar{0} + \bar{A}0 = A$
- $A \oplus 1 = A \cdot \bar{1} + \bar{A} \cdot 1 = 0 + \bar{A} = \bar{A}$
- $A \oplus \bar{A} = A \cdot \bar{\bar{A}} + \bar{A} \cdot \bar{A} = A + \bar{A} = 1$
- $A \oplus A \oplus A \ldots\ldots\ldots \oplus A = 0$ if no. of terms is even
  $= A$ if no. of terms is odd
- $A \oplus 0 \oplus 0 \ldots\ldots\ldots \oplus 0 = A$
- $A \oplus 1 \oplus 1 \ldots\ldots\ldots \oplus 1 = \bar{A}$ if no. of terms is even
  $= A$ if no. of terms is odd
- $\bar{A} \oplus \bar{A} \oplus \bar{A} \ldots\ldots\ldots \oplus \bar{A} = 0$ if no. of terms is even
  $= \bar{A}$ if no. of terms is odd
- $\bar{A} \oplus B = \bar{\bar{A}}B + \bar{A}\bar{B}$
  $= AB + \bar{A}\bar{B}$
  $= A \odot B$

  i.e. one of the inputs of EX–OR is inverted then the EX–OR gate works as EX–NOR.
- $A \oplus B \oplus AB = A + B$
- $A \oplus B \oplus 1 = \overline{A \oplus B} = A \odot B$

**The Exclusive–NOR Gate (EX–NOR)**

The EX–NOR is modified form of NOR gate.




Standard Logical Symbol for 2–input EX–NOR Gate

EX–NOR function is represented by the symbol $\odot$

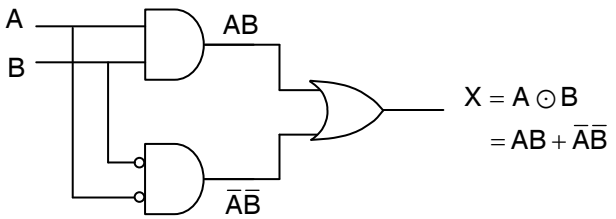
EX–NOR is mathematically represented as

$$X = A \odot B$$
$$= AB + \overline{A}\,\overline{B}$$

Truth Table

| INPUTS | | OUTPUT |
|---|---|---|
| A | B | X |
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

For EX–NOR Gate, for even inputs output is HIGH and for odd inputs output is LOW.




EX–NOR is called as equality comparator as output of EX–NOR is logic 1 only when the even no. of inputs are equal.

*Useful formulae for EX–NOR Gate*

- $A \odot 0 = \overline{A}$
- $A \odot 1 = A$
- $A \odot A = 1$
- $A \odot \overline{A} = 0$
- $\overline{A} \odot B = A \oplus B$
- $A \odot \overline{B} = A \oplus B$

- $\overline{A \odot B} = A \oplus B$
- $A \odot B \odot AB = A + B$
- $A \odot AB = A\overline{B}$
- $A \odot (A + B) = \overline{A}B$
- $A \odot \overline{A}B = \overline{A + B}$
- $A \odot (\overline{A} + B) = \overline{A\overline{B}} = AB$

## KARNAUGH MAP (K–MAP)

Karnaugh Map (K-map) is used to determine minimal expression. Each n variable map consists of $2^n$ cells or squares.

A Three variable map

| C \ AB | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | 0 | 2 | 6 | 4 |
| 1 | 1 | 3 | 7 | 5 |

A Four variable K–map

| CD \ AB | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 0 | 4 | 12 | 8 |
| 01 | 1 | 5 | 13 | 9 |
| 11 | 3 | 7 | 15 | 11 |
| 10 | 2 | 6 | 14 | 10 |

A K–map provides a systematic method for simplifying the Boolean expressions and, if properly used, will produce the simplest SOP or POS expression possible, known as the minimum expression.

*Cell Adjacency*

The cells in a K–map are arranged so that there is only a single–variable change between adjacent cells. Adjacency is defined by a single variable change.



The K–map is an array of cells in which each cell represents a binary value of the given input variables.

**K–Map for SOP Function**

*Canonical form*

The switching function expressed as the sum of all the minterms is called the canonical Sum Of Products (SOP) or disjunctive normal expression.

*Minterm*

'Minterm' is a product term which has each of all the variables as factors in either complemented or uncomplemented form.

**To obtain minimum SOP expression using K–map**

*Procedure*

Step 1 : Grouping the 1s : A group must contain either 1, 2, 4, 8 or 16 cells, which are all power of two. In the case of 3–variable map, $2^3 = 8$ cells in the maximum group.

Step 2 : Each cell in a group must be adjacent to one or more cells in that same group, but all cells in the group do not have to be adjacent to each other.

Step 3 : Always include the largest possible number of 1s in a group accordance with rule 1.

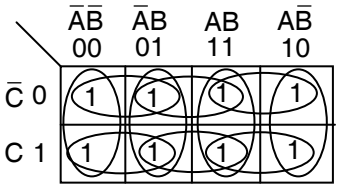
Step 4 : Each 1 on the map must be included in at least one group.

Step 5 : The 1s already in a group can be included in another group as long as overlapping groups include noncommon 1s.

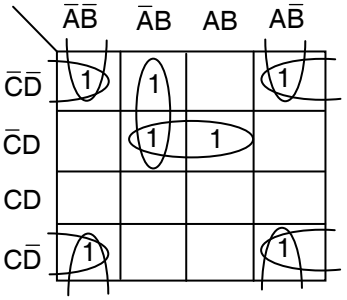
**Grouping of K–map**

a) *Two–one's grouping*

- On 3–variable K–map

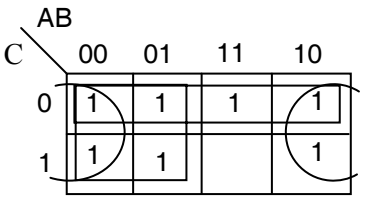



- On 4– variable K–map

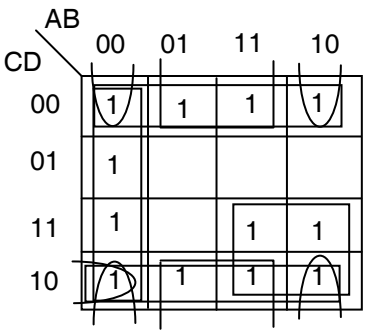



b) *Four–one's grouping*

- On 3– variable K–map

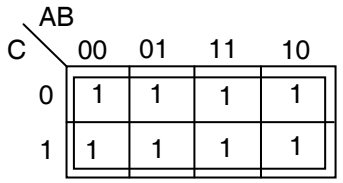



- On 4– variable K–map

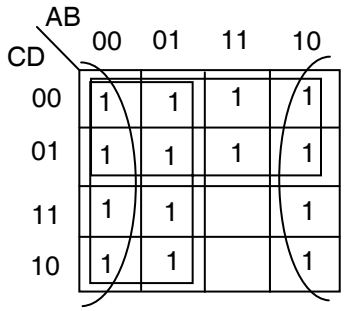


c) *8–one's grouping*

- On 3– variable K–map




- On 4– variable K–map




*SOP form of switching function from Truth table*

Truth Table

| INPUTS | | | OUTPUT |
|---|---|---|---|
| A | B | C | X |
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 1 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 0 |
| 1 | 1 | 1 | 1 |

From truth table it is seen that X(A, B, C) is '1' for 1, 2, 3, 5, 7 decimal values of inputs.

$$X(A, B, C) = \sum(1, 2, 3, 5, 7)$$

$$= 001 + 010 + 011 + 101 + 111$$

$$= \bar{A}\bar{B}C + \bar{A}B\bar{C} + \bar{A}BC + A\bar{B}C + ABC$$

which is a SOP form of function.

**K–Map for POS Function**



The process for minimizing a POS expression is same as for an SOP expression except that 0s are to be grouped to produce minimum sum term instead of grouping 1s to produce minimum product term.

- *3 – variable K–map*

| C \ A+B | 0+0 (A+B) | 0+1 (A+$\bar{B}$) | 1+1 ($\bar{A}$+$\bar{B}$) | 1+0 ($\bar{A}$+B) |
|---|---|---|---|---|
| 0 (C) | 0 | 2 | 6 | 4 |
| 1 ($\bar{C}$) | 1 | 3 | 7 | 5 |

- *4 – variable K–map*

| C+D \ A+B | 0+0 (A+B) | 0+1 (A+$\bar{B}$) | 1+1 ($\bar{A}$+$\bar{B}$) | 1+0 ($\bar{A}$+B) |
|---|---|---|---|---|
| 0+0 (C+D) | 0 | 4 | 12 | 8 |
| 0+1 (C+$\bar{D}$) | 1 | 5 | 13 | 9 |
| 1+1 ($\bar{C}$+$\bar{D}$) | 3 | 7 | 15 | 11 |
| 1+0 ($\bar{C}$+D) | 2 | 6 | 14 | 10 |

*Canonical form*

The switching function expressed as the product of all the maxterms is called the canonical Product Of Sum (POS) or disjunctive normal expression.

*Maxterm*

'Maxterm' is a sum term which has each of all the variables as factors in either complemented or uncomplemented form.

The POS form is the products of those sum combinations for which the function has the value '0'.

The canonical SOP form for the expression given below :

$$X(A,B,C) = AB + \bar{A}B + BC + B\bar{C}$$

$$= AB(C+\bar{C}) + \bar{A}B(C+\bar{C}) + BC(A+\bar{A}) + B\bar{C}(A+\bar{A})$$

$$= ABC + AB\bar{C} + \bar{A}BC + \bar{A}B\bar{C} + ABC + \bar{A}BC + A\bar{B}C + \bar{A}\bar{B}C$$

$$= ABC + AB\bar{C} + \bar{A}BC + \bar{A}B\bar{C} + A\bar{B}C + \bar{A}\bar{B}C$$

$$= \sum(1,2,3,5,6,7)$$

For above expression in POS form is found out by complementing minterm equation.

i.e. POS form = Complement of $\sum(4)$

= Complement of $\sum(100)$

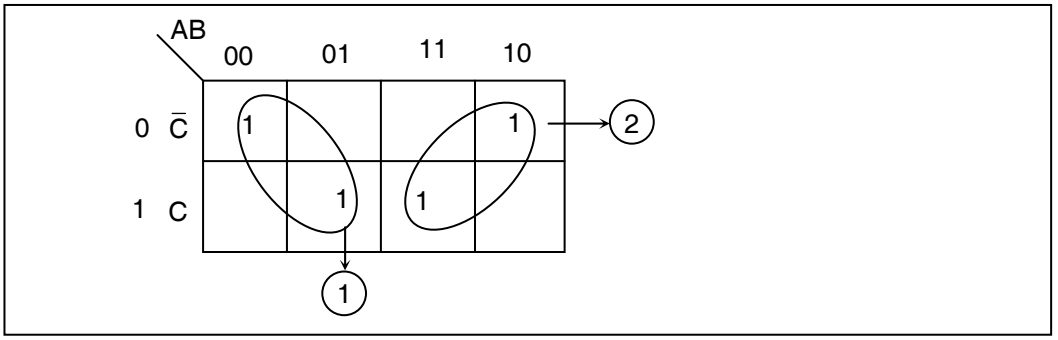
$= \bar{A} + B + C$

Similarly, if any particular variable does not occur in any sum term, then say for X(A, B, C) add $A\bar{A}, B\bar{B}$ or $C\bar{C}$ as the case may be. Then convert the sum terms into product of sums and eliminate the repeated terms.

**Diagonal and Offset Grouping**

If we have to construct the equation using EX–OR or EX–NOR gates then use the Diagonal and Offset grouping.

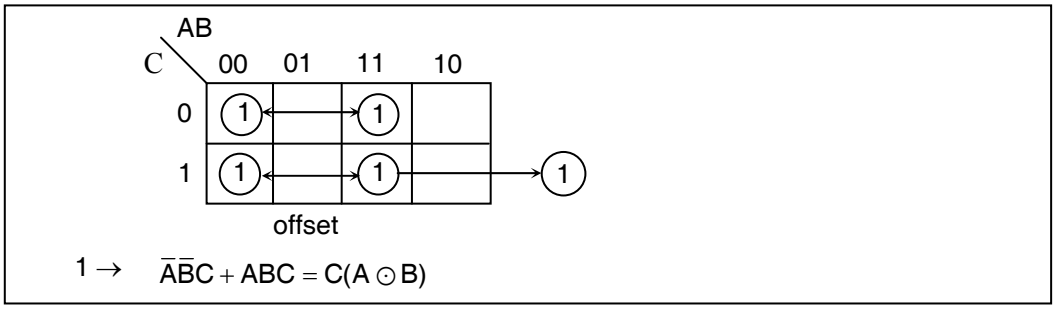
*Diagonal Grouping*




Example :

1. $\bar{A}\bar{B}\bar{C} + \bar{A}BC = \bar{A}(\bar{B}\bar{C} + BC) = \bar{A}(B \odot C)$
2. $A\bar{B}\bar{C} + ABC = A(B \odot C)$

*Offset Grouping*




Avoid the mixing of grouping (i.e.) horizontal, vertical with diagonal and offset.

**Don't Care Conditions [X]**

Sometimes, the function can assume either a '0' or '1' value for a number of combinations, under the situation when the variables are not mutually independent.



The combinations for which the value of the function is not specified with certainty are called don't care conditions.

## Conversion of Standard SOP to Standard POS

*Procedure*

Step 1: Evaluate each product term in the SOP expression i.e. determine the binary numbers that represent the product terms.

Step 2 : Determine all of the binary numbers not included in the evaluation is step 1.

Step 3 : Write the equivalent sum term for each binary number from step 2 and express in POS form.

Using a similar procedure POS form is converted into SOP form.

*Example*

$y = \Pi M(1, 3, 4, 6, 9, 11, 13, 15)$

Minimize the above equation using K–map and realize the reduced expression using

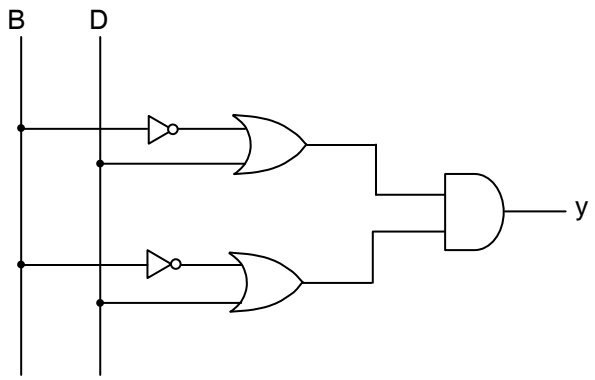
(i) Basic gates only
(ii) NOR gates only
(iii) NAND gates only

*Solution*

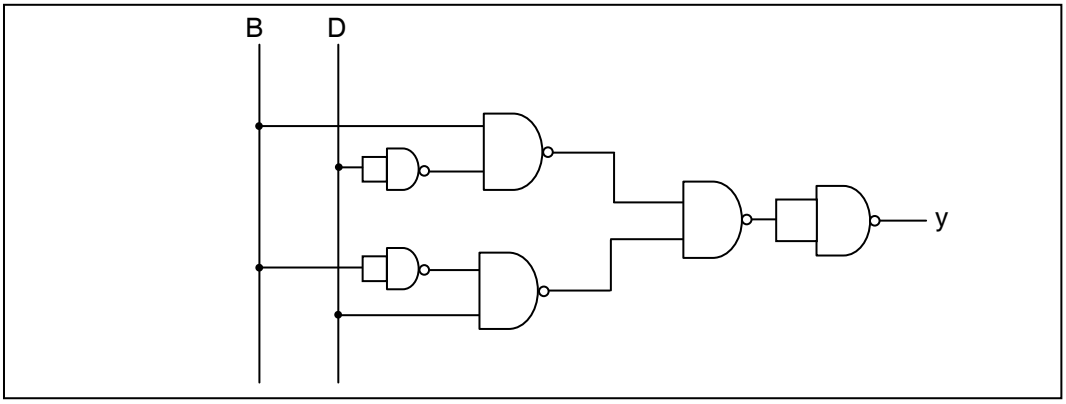
| CD \ AB | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 1 | 0 | 0 | 1 |
| 01 | 0 | 1 | 1 | 0 |
| 11 | 0 | 1 | 1 | 0 |
| 10 | 1 | 0 | 0 | 1 |

$y_{POS} = (\overline{B} + D)(B + \overline{D}) = B \odot D$

(i) Using basic gates



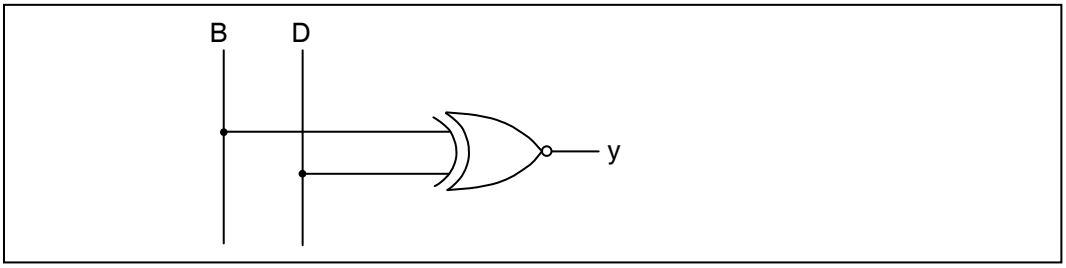
(ii) Using NAND Gates




(iii) Using NOR Gates

$y = (\overline{B} + D)(B + \overline{D})$

$= \overline{B}B + \overline{B}\overline{D} + BD + D\overline{D} = B \odot D$




## LIST OF FORMULAE

**Boolean Laws**

- *The complementation laws*

  $\overline{0} = 1$

  $\overline{1} = 0$

  $\overline{\overline{A}} = A$

- *Laws of Boolean Algebra for AND Function*

| | |
|---|---|
| $A \cdot 0 = 0$ | Null |
| $A \cdot 1 = A$ | Identity |
| $A \cdot A = A$ | Idempotent |
| $A \cdot \overline{A} = 0$ | Complement |
| $A \cdot B = B \cdot A$ | Commutative |
| $ABC = A(BC) = (AB)C$ | Associative |
| $A \cdot (B + C) = A \cdot B + A \cdot C$ | Distributive |

- *Laws of Boolean Algebra for OR Function*

| | |
|---|---|
| $A + 0 = 0$ | Null |
| $A + 1 = A$ | Identity |
| $A + A = A$ | Idempotent |
| $A + \overline{A} = 1$ | Complement |
| $A + B = B + A$ | Commutative law |
| $A + (B + C) = (A + B) + C$ | Associative law |
| $(A + B)(A + C) = A + B \cdot C$ | Distributive law |

- *De–Morgan's Laws*

  $\overline{A + B} = \overline{A} \cdot \overline{B}$

  NOR = bubbled AND

  $\overline{AB} = \overline{A} + \overline{B}$

  NAND = bubbled OR

- *Absorption Laws*

  A(A + B) = A

  Similarly A + AB = A

**Some Rules of Boolean Algebra**

$\overline{\overline{A}} = A$ (i.e. the double complement of a variable is always equal to the variable.)

$A + \overline{A}B = A + B$

$\overline{A} + AB = \overline{A} + B$

$(A + B)(A + C) = A + BC$

**Exclusive–OR Gate (EX–OR)**

$X = A \oplus B$

| | |
|---|---|
| $X = \overline{A}B + A\overline{B}$ | ….SOP form |
| $X = (\overline{A} + B)(A + \overline{B})$ | ….POS form |
| $X = (A + B)(\overline{A} + \overline{B})$ | ….POS form |
| $X = \overline{AB + \overline{A}\overline{B}}$ | ….SOP form |
| $X = (A + B)(\overline{A \cdot B})$ | ….POS form |

*Useful formulae for EX–OR function*

$A \oplus A = A\overline{A} + \overline{A}A = 0$

$A \oplus 0 = A\overline{0} + \overline{A}0 = A$

$A \oplus 1 = A \cdot \overline{1} + \overline{A} \cdot 1 = 0 + \overline{A} = \overline{A}$

$A \oplus \overline{A} = A \cdot \overline{\overline{A}} + \overline{A} \cdot \overline{A} = A + \overline{A} = 1$

$A \oplus A \oplus A..... \oplus A = 0$ if no. of terms is even

$= A$ if no. of terms is odd

$A \oplus 0 \oplus 0..... \oplus 0 = A$

$A \oplus 1 \oplus 1..... \oplus 1 = \overline{A}$ if no. of terms is even

$= A$ if no. of terms is odd

$\overline{A} \oplus \overline{A} \oplus \overline{A}..... \oplus \overline{A} = 0$ if no. of terms is even

$= \overline{A}$ if no. of terms is odd

$\overline{A} \oplus B = \overline{\overline{A}}B + \overline{A}\overline{B}$

$= AB + \overline{A}\overline{B} = A \odot B$

i.e. one of the inputs of EX–OR is inverted then the EX–OR gate works as EX–NOR.

$A \oplus B \oplus AB = A + B$

$A \oplus B \oplus 1 = \overline{A \oplus B} = A \odot B$

**Exclusive–NOR Gate (EX–NOR)**

EX–NOR is mathematically represented as

$X = AB + \overline{A}\overline{B}$

*Useful formulae using EX–NOR Gate*

$A \odot 0 = \overline{A}$

$A \odot 1 = A$

$A \odot A = 1$

$A \odot \overline{A} = 0$

$\overline{A} \odot B = A \oplus B$

$A \odot \overline{B} = A \oplus B$

$\overline{A \odot B} = A \oplus B$

$A \odot B \odot AB = A + B$

$A \odot AB = \overline{A\overline{B}}$

$A \odot (A + B) = \overline{\overline{A}B}$

$A \odot \overline{A}B = \overline{A + B}$

$A \odot (\overline{A} + B) = \overline{\overline{A}\overline{B}} = AB$

## LMR (LAST MINUTE REVISION)

- **Number system**
  To subtract a smaller number from a larger number the 1's complement method is as follows:
  (i) Determine 1's complement of smaller number
  (ii) Add 1's complement to the larger number
  (iii) Remove the carry and add it to the result. This is called end around carry.

To subtract a smaller number from larger one the 2's complement method is as follows:

(i) Determine the 2's complement of smaller number
(ii) Add the 2's complement to the larger number
(iii) Discard the carry. (Always there is a carry)

- **Conversions**

*Binary to Decimal Conversion*

– A binary number is a weighted number. The value of a given binary number in terms of its decimal equivalent can be determined by adding the products of each bit and its weight.

– (Dibble Dabble Method)
  (i) Start with the MSB and multiply by 2.
  (ii) Add the next bit to the product.
  (iii) Multiply the sum by 2.
  (iv) Add the next bit to the sum.
  (v) Multiply by 2 and repeat the steps until all the bits are exhausted.

*Decimal to Binary Conversion*

Sum of weight method – by placing 1 in the appropriate weight position and 0 in other position.

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| Remember: … | $2^5$, | $2^4$, | $2^3$, | $2^2$, | $2^1$, | $2^0$ | |
| … | 32 | 16 | 8 | 4 | 2 | 1 | Binary weights. |

*Octal to Decimal Conversion*

| Weight | $8^3$ | $8^2$ | $8^1$ | $8^0$ |
|---|---|---|---|---|
| Decimal value | 512 | 64 | 8 | 1 |
| Octal No. | 2 | 3 | 7 | 4 |

*Decimal to Octal Conversion*

To get octal equivalent of a decimal number we use repeated division by 8. The decimal number is divided by right, the quotient is divided by 8 and the remainders obtained will give the octal number. The first remainder is the LSB.

*Octal to Binary Conversion*

To convert octal to binary, simply replace the octal digit by its binary number using 3 bits.

The hexadecimal system has a base of 16 i.e. it is composed of 16 digits and characters. Ten digits and six alphabetic characters make up this system 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F.

*Hexadecimal to Binary Conversion*

Replace each hexadecimal digit by the four bit binary number.

*Decimal to Hexadecimal Conversion*

Repeated division of decimal number by 16 gives the hexadecimal number, formed by the remainders of each division. The $1^{st}$ remainder is the LSB of the hexadecimal number.

- **8–4–2–1 (BCD) Code**

Decimal digits 0 through 9 are represented by their binary equivalents using four bits.

- **Excess-3 Code**
  The code can be derived from the BCD by adding 3 to each coded number.
- Boolean algebra uses two binary operations '+' (plus) and '.'(dot) and one unary operations '/'(complement) operation.
- Variable : A variable is a symbol used to represent a logical quantity, can have a value 1 or 0.
  Complement : It is the inverse of a variable.
  Literal : It is a variable or the complement of a variable
- There are three types of basic logic Gates : AND, OR, NOT.
- In inverter, when the input is LOW, the output is HIGH; when the input is HIGH, the output is LOW. The bubble always indicate inversion in digital circuit.
- In AND gate, the output is HIGH only if both the inputs are HIGH.
- An OR gate produces a HIGH on the output when any of the inputs is HIGH.
- Boolean addition is the same as the OR function.
- Boolean addition differs from binary addition in the case where two 1's are added. There is no carry in Boolean addition.
- When solving Boolean expressions, the precedence in descending order is
  a. Parenthesis b. NOT
  c AND d. OR
- De–Morgan's Theorem : It states that the complement of a function is obtained by interchanging AND and OR operators and complementing each literal.
- The NAND and NOR gates are called universal gates as any basic gate (AND, OR and NOT) can be implemented using these gates.
- NAND gate is equivalent to bubbled OR gate.
- The output of NAND gate is HIGH, when at least one input to the NAND gate is LOW.
- NOR gate is equivalent to bubbled AND gate.
- The output of NOR gate is HIGH only when all the inputs are LOW.
- The NAND and NOR functions are commutative but not associative.
- De–Morganization : "just break the line, change the sign".
- Converting circuits to Universal Logic :
  a. Draw the circuit in AND/OR/invert logic
  b. If NAND realization is chosen, add a circle to the outputs of each AND gate on the logic diagram, and add circles to the inputs of all OR gates.
  c. In NOR realization, add circle to the output of gates and add circle to the inputs of AND gates.
  d. Add or subtract an inverter on each line that received a circle in step 2 or 3.
- In EX–OR, for even inputs output is LOW and for odd inputs output is HIGH.
- It is also noted that EX–OR is inequality comparator.
- EX–NOR is called as equality comparator as output of EX–NOR is logic 1 only when the even no. of inputs are equal.

- K–map is used to determine minimal expression. Each n variable map consists of $2^n$ cells or squares.
- Canonical form : The switching function expressed as the sum of all the minterms is called the canonical Sum Of Products (SOP) or disjunctive normal expression.
- Minterm : 'Minterm' is a product term which has each of all the variables as factors in either complemented or uncomplemented form.
- Maxterm : 'Maxterm' is a sum term which has each of all the variables as factors in either complemented or uncomplemented form.
- The switching function expressed as a product of all the Maxterms is called the canonical Product Of Sum (POS) form.
- POS form is found out by complementing minterm equation.

❑ ❑ ❑ ❑ ❑ ❑

# ASSIGNMENT – 1

**Duration : 45 Min.** **Maximum Marks : 30**

**Q1 to Q6 carry one mark each**

1. $(1101101 . 1011)_2 = (\quad)_{10}$
   (A) 109.6875 (B) 109.7865
   (C) 109.6880 (D) 119.08

2. The equivalent in decimal for the excess–3 code 843 is
   (A) 510 (B) 840
   (C) 846 (D) None of these

3. Find odd man out
   i) AA = A ii) $1 + A = 1 + \bar{A}$
   iii) A + BC = (A + B) (A + C) iv) $\bar{A} + AC = A + C$
   (A) (ii) and (iv) (B) (ii), (iii) & (iv)
   (C) (i) & (iv) (D) (iv) only

4. Inhibitor is also known as
   (A) AND–NOT (B) OR–NOT
   (C) Strobe (D) All of above

5. Subtraction of the given signed number is
   00001100 – 11110111
   (A) 00010101 (B) 11101010
   (C) 00001011 (D) 00110101

6. Determine the decimal values of the binary numbers in 2's complement
   Number is : 10101010
   (A) + 86 (B) – 86
   (C) + 98 (D) –98

**Q7 to Q18 carry two marks each**

7. Given Boolean equation as
   $y = (A, B, C, D)$
   $y = \Pi M(0, 1, 2, 3, 6, 7, 8, 9, 11, 15)$
   The minimized POS form of above equation is
   (A) $y_{POS} = (A + B)(A + \bar{D})(\bar{C} + \bar{D})(B + D)$
   (B) $y_{POS} = (\bar{A} + \bar{B})(\bar{A} + D)(D + C)(\bar{B} + \bar{D})$
   (C) $y_{POS} = (A + \bar{B})(A + \bar{D})(C + \bar{D})(\bar{B} + D)$
   (D) $y_{POS} = (A + B)(A + D)(C + D)(B + D)$

8. The number of NOR gates required to implement NAND gate is
   (A) 3 (B) 4
   (C) 5 (D) 2

9. To represent reduced expression of the following POS form, the no. of NOR gates required are

$y = \Pi M(0, 1, 2, 3, 6, 7, 8, 9, 11, 15)$

(A) 6 (B) 7
(C) 8 (D) 9

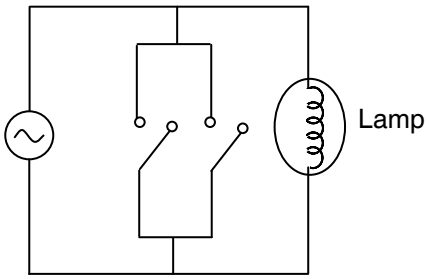
10. The no. of required gates to represent XOR using NAND only are

(A) 3 (B) 4
(C) 4 (D) 6

11. The decimal number –39 is expressed in 2's complement form as

(A) 00100111 (B) 11011000
(C) 11011001 (D) 11010001

12. The switching diagram depicts

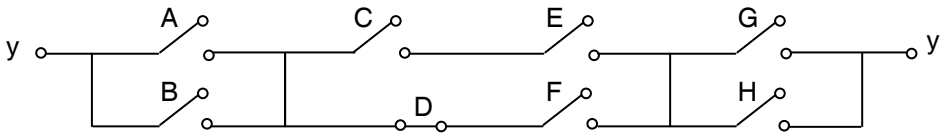



(A) OR gate (B) NOR gate
(C) AND gate (D) NAND gate

13. $\overline{(A+\bar{B})(\bar{\bar{A}}+B)} =$

(A) $\bar{A}+B$ (B) $A+\bar{B}$
(C) A (D) B

14. For the switching circuit given below, taking open as 0 and close as 1, the expression for the circuit is




(A) $y = (A+B)(CE+\bar{D}F)(G+H)$

(B) $y = (A+B)(C+E+\bar{D}+F)(G+H)$

(C) $y = AB+C\bar{D}+EF+GH$

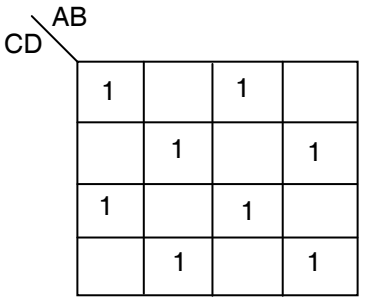
(D) None of these

15. The given expression can be reduced to

$y = \overline{\overline{A\bar{B}+ABC}+A(B+A\bar{B})}$

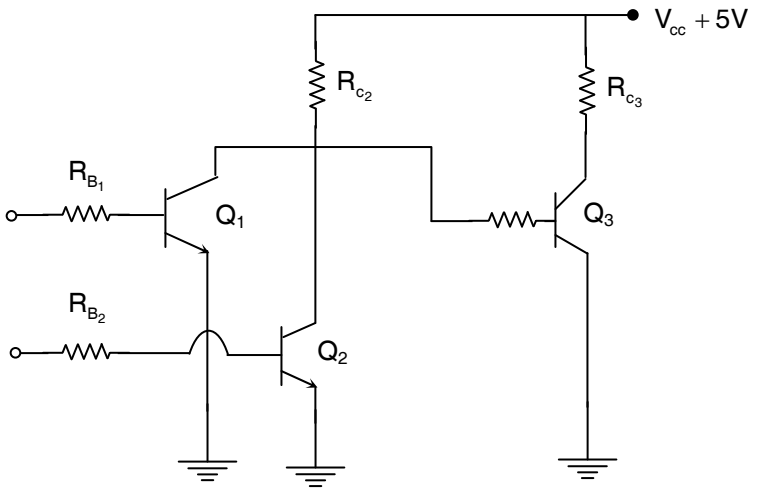
(A) 1 (B) 0
(C) A (D) $\overline{AB}$

16. To realize minimized function of K – map shown below, no. of gates required are




(A) 1 (B) 3
(C) 4 (D) None of these

17. The following diagram represents which type of gate?




(A) AND (B) NAND
(C) OR (D) NOR

18. The minimum number of NOR gates to implement the function f = AB + $\bar{A}B$ are
(A) 2 (B) 4
(C) 10 (D) 6

❑ ❑ ❑ ❑ ❑ ❑

# TEST PAPER – 1

**Duration : 30 Min.** **Maximum Marks : 25**

**Q1 to Q5 carry one mark each**

1. If one of the input to an _____ gate is inverted then it becomes an INHIBITOR.
   (A) AND (B) NAND
   (C) NOR (D) XOR

2. $[(A + \overline{A}B)(A + \overline{A}\,\overline{B})][(CD + \overline{C}\,\overline{D}) + (C \oplus D)] =$
   (A) B (B) A
   (C) 0 (D) 1

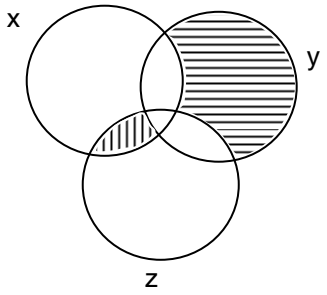
3. Which one of the following is equivalent to OR–AND realization?
   (A) NAND–NOR realization (B) NAND–NAND realization
   (C) NOR–NOR realization (D) NOR–NAND realization

4. Which of the following gates is a series circuit gate ?
   (A) AND gate (B) OR gate
   (C) XOR gate (D) XNOR gate

5. The maxterm designator of the term
   $\overline{A} + B + \overline{C} + D$ is
   (A) 10 (B) 05
   (C) 13 (D) None of these

**Q6 to Q13 carry two marks each**

6. The Boolean expression for shaded area in the Venn diagram is




   (A) $\overline{(x + y) + \overline{x}\,\overline{y}\,\overline{z}}$ (B) $\overline{x} + \overline{z} + \overline{(x + y + z)}$
   (C) $(\overline{\overline{x} + \overline{z}}) + (x + y + z)$ (D) $(\overline{\overline{x} + \overline{y}}) + (x + y + z)$

7. The floating point binary number is given as

| S | E | F |
|---|---|---|
| 1 | ⊕ | 10001110001000000000000 |

Find the binary value of given number.

(A) 1100011000110000000 (B) −1100011000110000000
(C) −1100011100010000000 (D) 1100011100010000000

8. For the following K−map, POS form equation is

| CD \ AB | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 1 | 0 | 0 | 1 |
| 01 | 0 | 1 | 1 | 0 |
| 11 | 0 | 1 | 1 | 0 |
| 10 | 1 | 0 | 0 | 1 |

(A) $\prod$M(1, 3, 4, 6, 9, 11, 12, 14) (B) $\prod$M(0, 2, 5, 7, 8, 10, 11, 15)
(C) $\prod$M(1, 3, 4, 7, 8, 10, 11, 15) (D) $\prod$M(0, 2, 4, 7, 9, 11, 12, 14)

9. How many minimum NAND gates are required to implement following Boolean function ?

$$y = \overline{(A + \overline{B}) \cdot (\overline{A} + C)}$$

(A) 5 (B) 6
(C) 7 (D) 8

10. $(NOR) \cdot (XOR) \cdot (NAND) =$

(A) NOR (B) NAND
(C) XOR (D) XNOR

11. Simplify this expression AB + (B + C)A + B(B + C)

(A) B + AC (B) A + BC
(C) A + B (D) B +C

12. The single−precision floating point binary number of the binary no. 111111011100000 is

(A) 0 10001101 11111011100000000000000
(B) 1 10001001 11111011110000000000000
(C) 0 10110101 11111011100100000000000
(D) 1 11001010 11011001110010000000000

13. $A = x\,\overline{y}\,\overline{z} + \overline{x}\,y\,\overline{z} + \overline{x}\,\overline{y}\,z + x\,y\,z$ To implement the above Boolean expression, minimum no. of gates required are

(A) 1 (B) 2
(C) 3 (D) 4

**Q14(a) and Q14(b) carry two marks each**

**Linked Answer Question**

14(a). 1's complement of 27 is _____

(A) 10101 (B) 00010

(C) 00100 (D) 00110

14(b). It's 2's complement will be _____

(A) 00101 (B) 10110

(C) 00111 (D) 000111

❑ ❑ ❑ ❑ ❑ ❑